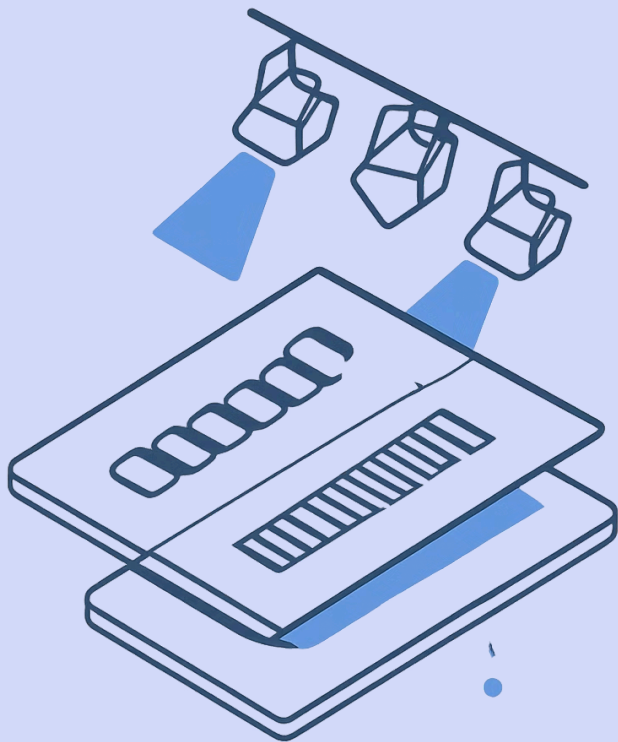




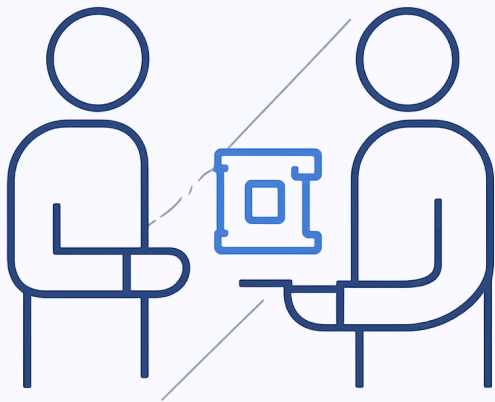
Grupo 1

Sistema de Venda de Ingressos

Integrantes: Amir Dantas, Gabriel Campos, Maria Lacerda, Pedro Corrêa, Thiago Santos, Willian Miranda

Três partes: o que é o projeto, o que foi apresentado da primeira vez, e o que foi feito desde então.

O Que É o Projeto

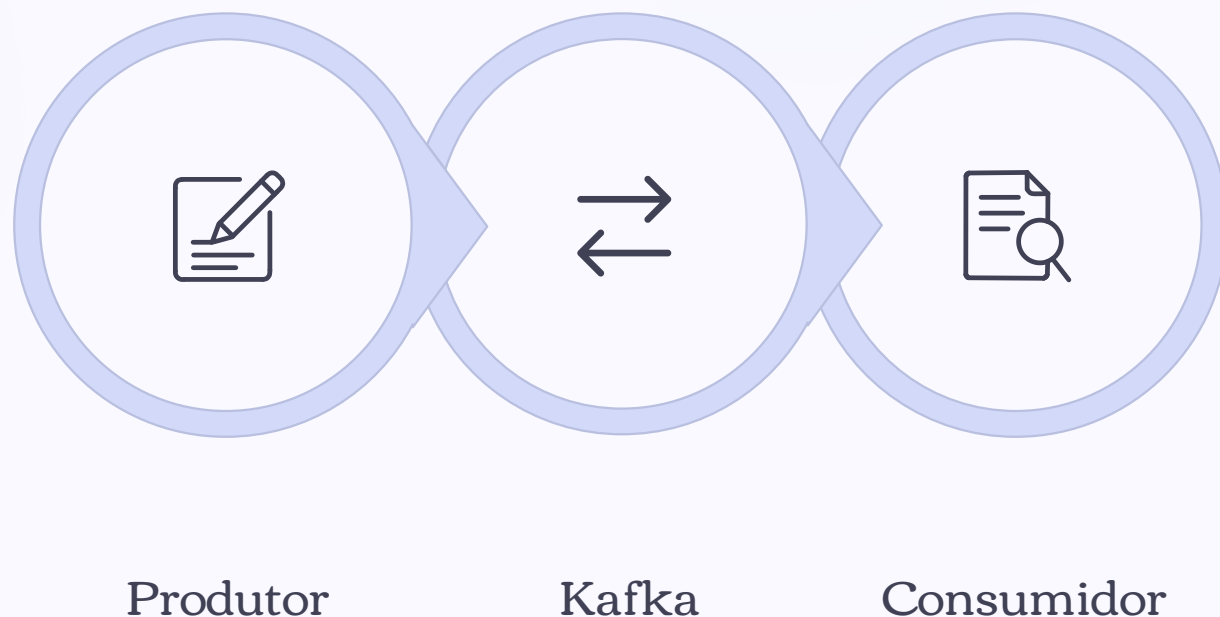


Reserva → Pagamento → Compensação

Comprador reserva ingressos; sistema valida limite por CPF e disponibilidade, publica **"ingresso reservado"** e aciona gateway de pagamento simulado. Se recusado ou expirado, ingresso volta ao estoque por compensação.

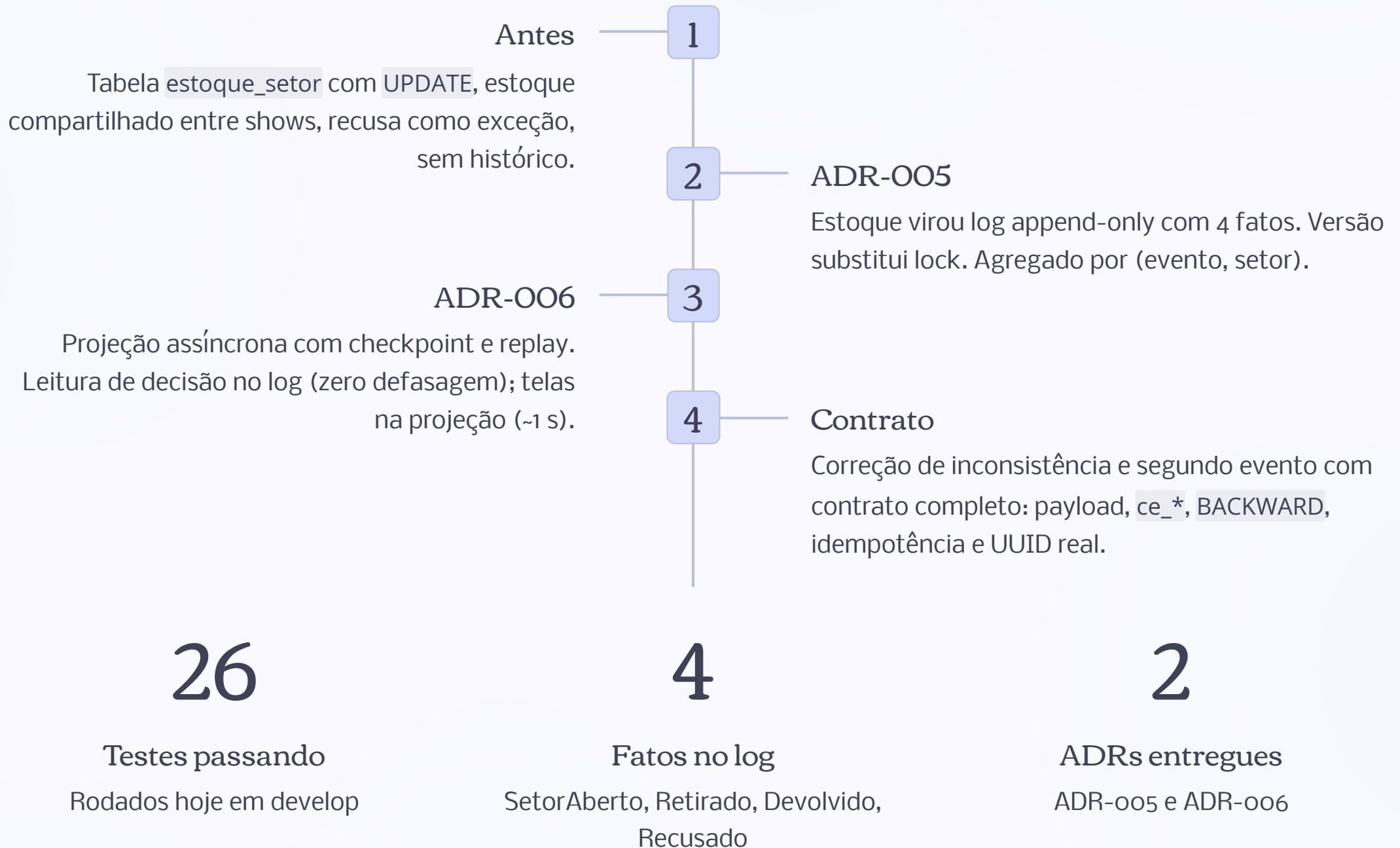
- ❏ Escolhido pelo ADR-002 e retomado no ADR-005: o intervalo entre reservar e pagar é onde sistemas de ingresso falham sob concorrência – dois compradores disputando o mesmo assento e o Event Sourcing para solucionar.

Arquitetura: Dois Serviços + Kafka



Dois serviços Maven independentes, sem POM pai. Kafka com CloudEvents 1.0 binário (`ce_*` nos cabeçalhos), chave de partição = evento – reservas do mesmo show ficam na mesma partição porque disputam o mesmo estoque.

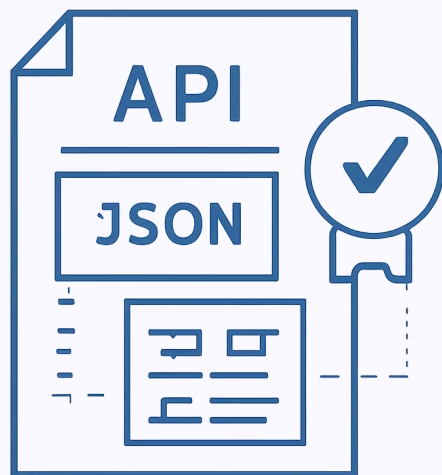
Resumo: O Que Mudou



O Contrato: CloudEvents 1.0 e Compatibilidade

`docs/contrato.md` – Seções 9-14

O contrato define o acordo entre produtor e consumidor usando **CloudEvents 1.0** em **binary mode**, separando metadados do evento e dados de negócio de forma explícita.



- Metadados ficam nos cabeçalhos do Kafka (`ce_*`) e os dados de negócio seguem no payload no formato JSON.
- A compatibilidade `BACKWARD` e os formatos de data estão documentados no contrato.
- A estratégia de partition key e a definição dos campos do evento também estão especificadas.
- `ce_id` identifica a mensagem CloudEvent; `eventid` identifica o evento de negócio.
- `eventid` agora usa formato `string UUID`, com exemplos reais para eliminar a inconsistência anterior.
- O foco é a evolução de `vendas.ingresso.reserva-compensada.v1`.

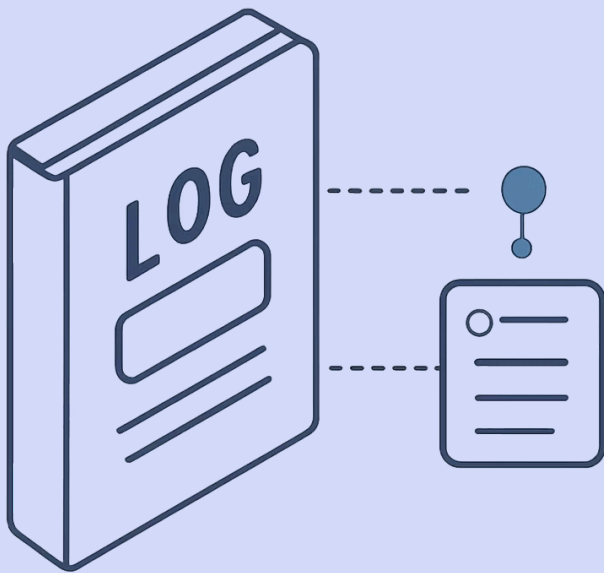
Como Era o Estoque – O "Antes"



Tabela `estoque_setor`

- Uma linha por setor, `UPDATE` por reserva
- Chave só `setor`: PISTA de **qualquer** show dividia os mesmos 100 ingressos
- Concorrência: `UPDATE ... WHERE quantidade_disponivel >= ?`
- Sem vaga → exceção, sem ack, mensagem voltava para sempre
- Nunca gravava a recusa – demanda além da oferta era irresponsável

ADR-005 – O Estoque Virou um Log



A tabela dizia o saldo e apagava a história. Trocamos por um log em que a única escrita é `INSERT`: o saldo passa a ser o resultado de reler os fatos.

Agregado: `EstoqueDoSetor`

Stream por **(evento, setor)** – Id `SHOW-PUCMINAS-2026:PISTA`.

Invariante resolvida dentro de um setor; chave de partição `evento` levada um nível abaixo. Corrige o estoque compartilhado entre shows.

Versão Substitui o Lock

Dois processos leem a PISTA na v7, tentam gravar v8. Banco aceita o primeiro `INSERT` e recusa o segundo por chave única → `ConcorrencaNoStreamException`. Sem lock, sem `SELECT FOR UPDATE`.

Os Quatro Fatos do Estoque



SetorAberto

Carrega **capacidade**. Primeiro evento de todo stream; substitui a semente do `schema.sql`.



IngressoRetirado

Carrega **quantidade** e `origemEventold`. N ingressos saíram.



IngressoDevolvido

Carrega **quantidade**, `origemEventold` e `motivo`.
Compensação do ADR-002 – anula sem apagar o fato anterior.



ReservaRecusada

Carrega **quantidadePedida**, `disponivelNoMomento` e `origemEventold`. Fato, não exceção: mede demanda além da oferta.

`EstoqueEvent` é `sealed` – switch exaustivo. Tipo gravado é nome de domínio ("`IngressoRetirado`"), não nome de classe: renomear a classe não invalida o log.

Projeção: Snapshot e Ponteiro

Separando a Fonte da Verdade do Modelo de Leitura

```
CREATE TABLE IF NOT EXISTS disponibilidade_por_setor (  
  evento      VARCHAR(80) NOT NULL,  
  setor       VARCHAR(50) NOT NULL,  
  capacidade  INT NOT NULL,  
  retirados   INT NOT NULL,  
  disponivel  INT NOT NULL,  
  PRIMARY KEY (evento, setor)  
);
```

disponibilidade_por_setor

O Snapshot

Tabela de leitura otimizada que reflete o estado atual acumulado do log de eventos.

Guarda Capacidade, Retirados e Disponível por evento/setor.

Evita recomputar todo o log de eventos a cada consulta do usuário.

```
CREATE TABLE IF NOT EXISTS projecao_checkpoint (  
  projecao      VARCHAR(60) PRIMARY KEY,  
  ultima_sequencia BIGINT NOT NULL  
);
```

projecao_checkpoint

O Ponteiro Global

Marcador de posição que armazena a última sequência do log já processada.

Um job agendado (Scheduled) verifica a cada 1 segundo se há novos eventos após a sequência atual.

Lê e atualiza os dados em lotes (batching) de forma assíncrona.

Resiliência e Reconstrução do Estado

Recuperação Determinística a partir da Fonte da Verdade

- ❑ **Event Sourcing garante resiliência:** se a tabela de snapshot for corrompida ou apagada, nenhum dado de negócio é perdido.

Fluxo Sequencial de Reconstrução

1 Limpeza Total

Exclui todos os registros corrompidos ou antigos da tabela `disponibilidade_por_setor`, assegurando um estado inicial limpo.

2 Reset do Ponteiro

Define a sequência do `projecao_checkpoint` de volta para o, preparando para uma releitura completa do histórico.

3 Re-leitura Histórica

Lê o histórico completo da tabela `evento_do_estoque` em lotes de 500 registros, aplicando cada fato sequencialmente.

4 Sincronização Final

Re-aplica os eventos até que o checkpoint alcance o último evento do log, reconstruindo o estado completo e consistente do snapshot.

Garantia de Idempotência: Executar o reprocessamento N vezes gera exatamente o mesmo snapshot final.